Emergency Dispatch Priority and Coordination System

Project Report

ENSE707 Software Quality Assurance

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

Table of Content

1. Problem Definition and Approval

1.1 Background, Context and Stakeholders

The emergency dispatch systems answer emergency calls, record details about the emergencies, and dispatch the right units accordingly. This project is developed for Auckland City Council, who require a modern solution for managing emergency response services across Auckland. Even though it is modelled after a real system, it remains a simplified hypothetical model for the purpose of this project.

Auckland City Council is responsible for defining the requirements and in charge of the whole system. The main operational stakeholders in the modelled emergency dispatch system are the **Caller**, **Dispatcher**, **Field Responders**, and **Software Developers**. The system process flows from dispatcher getting information from the caller, then creating a case for which determines the unit type (ambulance, police, fire). Software developers are responsible for maintaining testing and improving the system.

The emergency dispatch software has significant practical relevance. Failing to respond to a case punctually, assigning the wrong priority/severity, or poor coordination of response units could lead to slower emergency responses. Because emergency dispatching is a life-critical process, the software that supports it must operate accurately and consistently under pressure.

1.2 Problem Statement and Software Quality Issues

Managing emergency dispatching requires all aspects of the system to be of high quality. Any compromises in quality, such as issues in **reliability, usability, or maintainability**, could potentially risk lives.

Auckland City Council's current emergency dispatch system relies on manual priority allocation by the dispatcher for incoming cases. This manual assignment could cause inconsistencies in priorities, resulting in low-priority cases being handled first and potential delays, **compromising the reliability** of the system.

Due to the system scale it becomes increasingly difficult to control which affects the dispatchers who do not have an efficient way to record and manage call logs. The dispatcher also has to directly communicate and manually dispatch units to an ongoing case. At the scale of an emergency dispatching system, the number of requests for units becomes overwhelming and hard to manage, **compromising usability**.

The system does not automatically log calls, meaning a manual process is used. The current system for callers to dispatchers sometimes goes down for a few seconds at a time during the day, which **undermines reliability** since some calls may not come through.

The current system consists of multiple dispatchers all working towards assigning cases to dispatched units. However, this is a **reliability issue** as it opens up potential race condition vulnerabilities where a dispatched unit may be assigned to multiple cases at once.

The system needs to be **robust and maintainable**, due to the constant need for system updates. The current design of the system has three dispatchable units: ambulance, police, and fire. If Auckland City Council were to add a new unit to the system, it would require significant code changes. Maintainability is important since neglecting it increases the complexity of adding new unit types into the system resulting in the increase of defects.

1.3 Why the problem is suitable for a Software Quality Assurance Project

Our selected problem is suitable for a Software Quality Assurance project because it contains clear software quality issues that can be analysed and tested. The main quality issues we mentioned in Section 1.2 are reliability, usability, and maintainability, which are also recognised quality characteristics within **ISO/IEC 25010**.

These issues are suitable for quality assurance because they can easily be used to form clear requirements, acceptance criteria, and test cases. This makes it possible to test the system to see if the quality issues have been improved or removed. This is important because any defects occurring on the system could deliver life threatening consequences.

2. Requirements and Quality Analysis

2.1 Functional Requirements

- **FR-01** The system shall provide an interface for the dispatcher to record call details, including caller information, incident notice, description of incident, and timestamp.
- **FR-02** The system shall allow the dispatcher to assign a case from a call.
- **FR-03** The system shall manage all dispatched units with each unit's current availability status (not available/available), unit type, location, identifier, and unit personnel count.
- **FR-04** The system shall automatically assign a priority level to each incoming emergency case based on its severity.
- **FR-05** The system shall automatically route a case to the appropriate emergency response department.
- **FR-06** The system shall automatically assign the most suitable unit within the relevant departments for a case.
- **FR-07** The system shall display all the cases on a single live dashboard.

- **FR-08** The system shall update the status of each case throughout the lifecycle of the case.
- **FR-09** The system shall automatically update a unit's availability status when it is assigned or the case is released.
- **FR-10** The system shall notify the assigned case with its details upon assignment.
- **FR-11** The system shall allow the dispatcher to mark a case (resolved/closed), which releases the unit assigned to it.
- **FR-12** The system shall allow the dispatcher to search through the history of logs by typing date, caller, and ID.

2.2 Non-Functional Requirements

Performance (P)
1. **NFR-P01**: When the dispatcher has assigned a case to the system, the system shall send it to the most suitable department for that case in under 2 seconds. 2. **NFR-P02**: The system shall allow at least 100 concurrent users without performance issues
Maintainability (M)
1. **NFR-M01**: The system shall be built using a modular architecture, so if one module fails, it will not cause the entire system to crash. 2. **NFR-M02**: The system shall be designed in a way that allows new user roles and departments to be added without requiring modification of existing classes.
Reliability (R)
1. **NFR-R01**: Data from a case shall not be lost when it is being sent from the dispatcher to the selected emergency department. 2. **NFR-R02**: The system shall run at least 99.9% of the time, given that it operates in a life-critical environment 24 hours a day.

3. **NFR-R03**: All case log entries and incident reports submitted through the system shall be saved without losing any of the data entered. 4. **NFR-R04**: When there is a large number of cases open simultaneously, the system shall continue to function without crashing.
Security (S)
1. **NFR-S01**: Any data that passes between users and the system must be encrypted to prevent unauthorised access. 2. **NFR-S02**: The system shall restrict access to system features based on each user's assigned role, as defined in section 1.1.
Usability (U)
1. **NFR-U01**: The system shall display a tailored view of the system for each user role, displaying only the tools and information required for that role. 2. **NFR-U02**: The system shall return all error messages in plain English so they can be easily understood. 3. **NFR-U03**: All users shall be able to operate the core functions of their role within the system after no more than 1 hour of training.

2.3 Requirements Analysis

The functional and non-functional requirements were analysed using the requirements quality criteria for **clarity, completeness, consistency, correctness, feasibility, and testability.** *The following table identifies the main issues found within the current requirements:*

Requirement	Quality Criteria	Analysis
FR-01	Clarity	The problem is the lack of clarity for the term “incident notice”.
FR-02	Clarity, Correctness	“Assign a case from a call” does not clearly describe the required action. The intended flow is the dispatcher ‘creating’ a case from a call.
FR-03	Clarity, Completeness, Testability	The word manage is a bit ambiguous in terms of what the system does with the unit’s information. Manage could mean just allowing the dispatcher to view, or does it also update the unit’s details?
FR-04	Completeness, Testability	The requirement states that severity determines priority, but the severity levels, priority levels and rules connecting them are not defined.
FR-05	Clarity, Completeness, Correctness, Testability	“Appropriate emergency response department” does not define how it is routed. The intended structure also allowed for a single case to be routed to multiple departments, but this functional requirement only refers to a single department.
FR-06	Clarity, Completeness,Testability	The phrase “most suitable unit” is ambiguous because it does not identify under what circumstances that a unit is the most suitable; it could be availability, proximity, experience, personnel count.
FR-07	Clarity, Completeness, Testability	The issue in this requirement is what’s displayed on the dashboard and what “live” means. It is clear that the system needs a dashboard to display all cases, but what exactly does each case display on the dashboard? Also, what live means could be a constant update or just updated when the page is refreshed.
FR-08	Completeness, testability	“Throughout the lifecycle” does not identify the lifecycle states or the events that cause status changes. It could be unassigned, in progress, completed, or delayed.

FR-09	Clarity, Consistency	The phrase "the case is released" is unclear because it does not define what "released" means. Does "released" mean that the case is no longer active for the unit, or does that the case is resolved? FR-11 instead states that the case is marked resolved/closed and the unit is released, so the two requirements also use inconsistent terminology for the same process.
FR-10	Clarity, Completeness, Correctness, Testability	"Notify the assigned case" does not identify a proper recipient. A case is not the intended notification recipient, making the stated behaviour incorrect.
FR-11	Clarity	This requirement clarifies what FR-09 does not elaborate on: while FR-09 states that the unit's availability updates when the case is released, it does not specify what action triggers the "release". The term "resolved/closed" is unclear because it does not specify whether resolved and closed are two separate case statuses or two terms for the same status.
FR-12	Completeness, testability	The requirement clearly explains that the system should allow the dispatcher to search through the history logs but is less clear on how the history logs are searched, even though date, caller, and ID are stated. It could be clearer on how this information contributes to finding the correct call log. For example, whether you need all this info to search for a log or just enter any of this info to get a list of logs.
NFR-P01	Completeness, Correctness, Testability	The requirement does not define how the "most suitable department" is determined. It also refers to a single department, while the proposed system allows a case to be routed to multiple relevant departments.
NFR-P02	Clarity, Completeness, Testability	"Performance Issues" is ambiguous. Without explicit details like acceptable response times or latency thresholds, it lacks clarity and completeness.

NFR-M01	Clarity, Testability	The terms "module" and "entire system to crash" are not clearly defined. Some may interpret the boundaries of a module or system failure differently, making it difficult to determine objectively whether the requirement has been satisfied.
NFR-M02	*No significant issues*	No significant issue identified.
NFR-R01	*No significant issues*	No significant issue identified.
NFR-R02	Clarity, Completeness, Testability	This requirement specifies a 99.9% uptime, which is measurable, but it lacks clarity and completeness regarding boundaries. It doesn't define whether planned maintenance work counts towards downtime, or over what timeframe the 99.9% uptime is measured (week, month, year), or what critical parts of the system must be active to count the system as "running".
NFR-R03	*No significant issues*	There is no significant issue with this requirement.
NFR-R04	Clarity, Completeness, Testability	In this requirement, the part "large number of cases" is ambiguous. Without a specific threshold, e.g. 1000 cases, the requirement is not complete or clear.
NFR-S01	*No significant issues*	No significant issue with this requirement.
NFR-S02	Completeness, Testability	This requirement establishes role-based access. However, testability and completeness rely on section 1.1. If this section definition of permissions is incomplete, it can introduce consistency gaps, preventing testers from validating the access boundaries.
NFR-U01	Completeness, Testability	The requirement mentions role-tailored views. However, it lacks completeness, missing the specific mappings per role.

NFR-U02	Clarity, Completeness, Testability	The term "plain English" in this requirement is subjective and can be interpreted differently for each person. This requirement lacks clarity and completeness because it doesn't define the standards or error-handling guidelines.
NFR-U03	Testability, Completeness	This requirement states a "1-hour training limit", which provides a clear metric, but it is incomplete on the evaluation parameters. It doesn't define what the core functions of their role are or what the success rate counts as "complete".

2.4 Improved and more testable requirements

Improved Functional Requirements

- **FR-01** The system shall provide an interface for the dispatcher to record an emergency call, requiring the caller information, incident type, incident location, incident description, timestamp, and relevant dispatch units required to go on the scene.
- **FR-02** The system shall allow the dispatcher to create an emergency case from a recorded call, capturing all the details provided in FR-01.
- **FR-03** The system shall allow each emergency response department to view an interface and update the availability status, unit type, location, identifier, and count of the response units managed by that department.
- **FR-04** The system shall automatically determine the priority level of a newly created emergency case using the predefined decision process for the relevant emergency response department, based on the incident information recorded by the dispatcher. The priority determination processes for Medical, Police, and Fire cases are defined in Appendix 1.
- **FR-05** The system shall automatically route a case to the emergency response departments based on which dispatch units were assigned to the case.

- **FR-06** Upon a case being routed to a department, the department's system shall automatically assign the units from that specific department based on whether the unit is available. If multiple units are available, the system shall assign the first unit in the department's unit list. If no units are available, the case shall remain in a queue until a unit becomes available.
- **FR-07** The system shall provide each emergency response department with a dashboard displaying the current cases assigned to that department. Any changes to the case information should be updated on the dashboard without needing the user to manually refresh.
- **FR-08** The system shall update the status of each case by following these states: "Open" when a case is created, not yet assigned to a unit, "In Progress" when at least one response unit is assigned and has not signed off, and "Closed" when all response units assigned to the case have signed off. If all assigned units are removed before completing the response, the case shall return to "Open".
- **FR-09** When a unit is assigned to an active case, the system shall change the unit's availability status to "not available". When that response unit signs off from the case after completing its response, the system shall change its status to "available".
- **FR-10** The system shall notify the relevant unit by a notification displayed on the dashboard of the system. The notification should contain the assigned case with its case details upon assignment.
- **FR-11** The system shall allow each response unit assigned to a case to sign off when it has completed its response. The system shall automatically close the case once all response units assigned to that case have signed off.
- **FR-12** The system shall allow the dispatcher to search the history of logs using any combination of date, caller name, or case ID, returning all logs that match at least one entered field.

Improved Non-functional Requirements

- **NFR-P01:** When a completed case is submitted for routing, the system should send the case to all relevant emergency response departments within 2 seconds.
- **NFR-P02:** The system shall support at least 100 concurrent users while maintaining a response time of 2 seconds or less when navigating between interfaces or refreshing a view.
- **NFR-M01**: The system shall be built using a modular architecture such that the core dispatch functions (case creation, unit assignment, status updates) are not affected by non-critical modules (notification, search logs). **Appendix 2**
- **NFR-M02**: The system shall be designed in a way that allows new user roles and departments to be added without requiring modification of existing classes.
- **NFR-R01**: Data from a case shall not be lost when it is being sent from the dispatcher to the selected emergency department.
- **NFR-R02**: The system should maintain at least 99.9% availability over a week; this includes the systems for Caller, Dispatcher, Field Responders, and Emergency Departments. Planned maintenance work does not count towards this downtime for any area.
- **NFR-R03**: All case log entries and incident reports submitted through the system shall be saved without losing any of the data entered.
- **NFR-R04**: When 100 cases are open simultaneously by the dispatchers, response units and emergency departments, the system shall continue accepting, displaying, and updating cases without crashing.
- **NFR-S01**: Any data that passes between users and the system must be encrypted to prevent unauthorised access.
- **NFR-S02**: The system shall restrict each authenticated user to only the system features and information permitted for their assigned role. The role permissions are defined in **Appendix 3**.
- **NFR-U01**: The system shall display a tailored interface of the system for each authenticated user's assigned role, displaying only the tools and information required for that role. The roles and role permissions are shown in **Appendix 3**
- **NFR-U02**: The system shall return all error messages in plain language so they can be easily understood. It should also provide an action that can resolve the error and let the user know

what to do. Technical error messages such as exceptions, stack traces, and debug information should never be displayed to the users.

- **NFR-U03**: After no more than 1 hour of training, each user shall be able to complete all defined core tasks for their assigned role without assistance. The core tasks for each role are listed in **Appendix 4**.

2.5 Defined acceptance criteria for the key features

These acceptance criteria are based on some core functionality features in the emergency dispatch systems

Requirement	Acceptance Criteria
FR-01	• Given a dispatcher is recording a call, when they submit the call record, all required details must be filled in. If any information is missing, the system shall reject the submission and inform what needs to be added. • Given a dispatcher is recording a call, when they submit without selected the dispatch unit type, the system should reject the submission
FR-02	• Given a dispatcher has completed recording the call, when they create a case from that record, then a new emergency case shall be created containing all the recorded details defined in FR-01. • Given a case is newly created from a recorded call, when creation completes, then the case status is set to “Open” with no units assigned to it yet.
FR-04	• Given a dispatcher has submitted a case, when the system determines its priority, it must apply the same decision processes from appendix 1 without manual input. • Given the incident infformation, when the decision process is applied, the assigned priority must match the outcome defined by that process.
FR-05	• Given that the dispatcher has created a case, when the case contains a specific unit type (police, fire, ambulance), then the case is automatically sent to the corresponding department (police to police department, fire to firefighter

	department, and ambulance to the Medical department). ● Given that the dispatcher has created a case with more than one dispatch unit type, when the case is ready for routing, then the case would automatically be sent to the corresponding departments.
FR-06	● Given a case has been routed to a department, when the department's system receives the case, then it shall automatically assign a unit from that department. ● Given a department has both available and unavailable units, when the system is assigning a unit, it should only assign units that are labelled "available". ● Given all units in a department are unavailable, when a case is routed to that department, then the case shall remain in the queue until a unit becomes available.
FR-07	● Given cases have been routed to a department, when a user opens that department's dashboard, then it shall display all cases that are assigned to that department, and no other cases. ● Given a department's dashboard is open, when the information of a case changes e.g. priority, location, then the dashboard should refresh and show the new changes automatically. ● Given a department's dashboard is open, when a case is routed to that department, then the dashboard should display the new case automatically.
FR-08	● Given a new case is created, when it has not been assigned to any unit, then the status of that case should be "open". ● Given a case has status "open", when it has been assigned to a unit, then the status should change to "in progress". ● Given a case has status "in progress", when some units have signed off but at least one assigned unit is still active, then the case shall remain "in progress" ● Given a case has status "in progress", when all the units assigned to that case have signed off, then the system shall update its status to "closed". ● Given a case has status "in progress", when all assigned units are removed before completing the response, then the case status should return to "open".
FR-09	● Given a response unit is labelled "available", when the unit is assigned to an active case, then the system shall change its availability status to "not available". ● Given a unit is currently assigned to a case, when the unit signs off, then the unit

	status would change from “not available” to “available”.
FR-11	• Given a response unit is assigned to a case, when the unit has completed its response, then the system shall allow the unit to sign off from that case. • Given a unit is currently assigned to a case with other units, when one unit signs off while at least one other unit remains active, then the case shall remain “in progress” • Given multiple units are currently assigned to a case,when the final remaining active unit signs off, then the case shall switch to “closed”

2.6 Identify relevant software quality attributes

The main software quality attributes relevant to the proposed system are reliability, usability, maintainability, performance, and security. Reliability, usability and maintainability relate directly to the main quality issues that were mentioned in Section 1.2.

The other quality issues; performance and security, are also important because the system will deal with users sensitive information and is time sensitive. These quality attributes are addressed in Section 2.2 and Section 2.4.

3. Proposed Solution

3.1 Proposed solution

The proposed project will develop a prototype Emergency Dispatch Priority and Coordination System. The system will include an interface for the dispatcher to record the details of each emergency call and create a case. Instead of the dispatcher manually handling every part of the dispatch process, the proposed system will standardise priority allocation and route each case to the relevant emergency response department or departments.

Additional stakeholders are introduced within the system. These departments include areas such as Medical, Police, and Fire. Once the case gets routed, the relevant department will be responsible for managing the case and coordinating its own response units. The system will also keep track of the status and availability of cases and units, allowing each of the respective departments to be informed at all times, so that they can manage them more clearly through a central interface. Response units assigned to the case sign off when they have completed their response. Once all assigned units have signed off, the system closes the case.

The system will also keep a history of previous cases, providing a complete audit trail of each emergency, so that important case information can be searched when needed. The overall design will be more modular so that new departments, response types, or user roles can be added more easily in the future.

3.2 How the proposed solution addresses the selected real world problem.

Our proposed solution focuses on reliability, usability and maintainability issues that are present in the current emergency dispatch process.

This solution addresses reliability issues by reducing the manual decision making required from the dispatcher. In the current system, they have to understand the information, figure out the priority, and do the process themselves. A standardised priority allocator can help by figuring out the priority and assigning it to the case automatically. Routing the cases to the relevant department also reduces the work needed from the dispatcher, thus reducing the risk for human error and incorrect case information.

The solution also solves usability issues by reducing the tasks the dispatcher manages manually. The main responsibility of the dispatcher will be recording emergency information. Everything else like priority allocation, routing, and management will be managed by the system and departments. This will reduce the overall workload the dispatcher does and makes the process easier for them.

Maintainability issues will be approached with a modular design. Different departments like call handling, priority allocation, department management, and unit coordination will be separated so that changes can be made easier. With a modular design, if a section needs changes or maintenance, the other parts of the system will remain untouched, reducing the chance for any new bugs. SOLID principles and design patterns will be used during development to support this.

4. Initial Prototype

4.1 Prototype Functionality

Our initial prototype includes the main dispatch workflow, where a dispatcher creates a case, and sends it to the relevant departments. When a call has come in, the dispatcher records information to create the case. The system will then store this case and apply the priority logic which was agreed on in Appendix 1. After the priority logic, the system will send the case to the requested departments (Fire, Police, and Medical). For each department, the system assigns the first suitable (availability) unit. If no unit is available, then the case goes in the queue until a unit can be assigned.

The prototype can also manage case and unit states. When a unit is assigned to a case, it becomes “unavailable”. They can sign-off later when the case is done, and the case will become “closed”, while the unit becomes “available again. If there are multiple units assigned to a case, then all units will have to sign-off and then the case becomes “closed”.

4.2 Role specific Interfaces

In the prototype, there are different interfaces that depend on the user’s role. Dispatchers can access the main dashboard, record new emergencies, and search through the history of cases. Department related users are restricted to their own department interfaces. For example, Medical, Police and Fire department users can see their own dashboards which show only that department’s cases.

If authorised, units can view their assignment information, availability, details of the incident, and notifications for any updates. Only they can sign-off their own cases, no-one else has access.

We implemented a prototype-only admin account to make visual testing and development more convenient. This role is not a part of the final system, and will be removed before release.

4.3 GUI Evidence

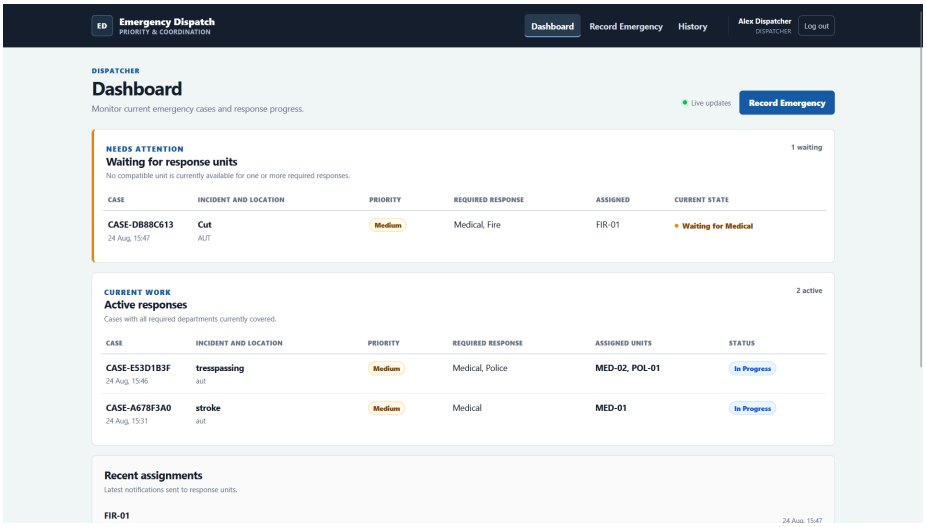


Figure 1. Dispatcher Dashboard showing active and waiting emergency cases.

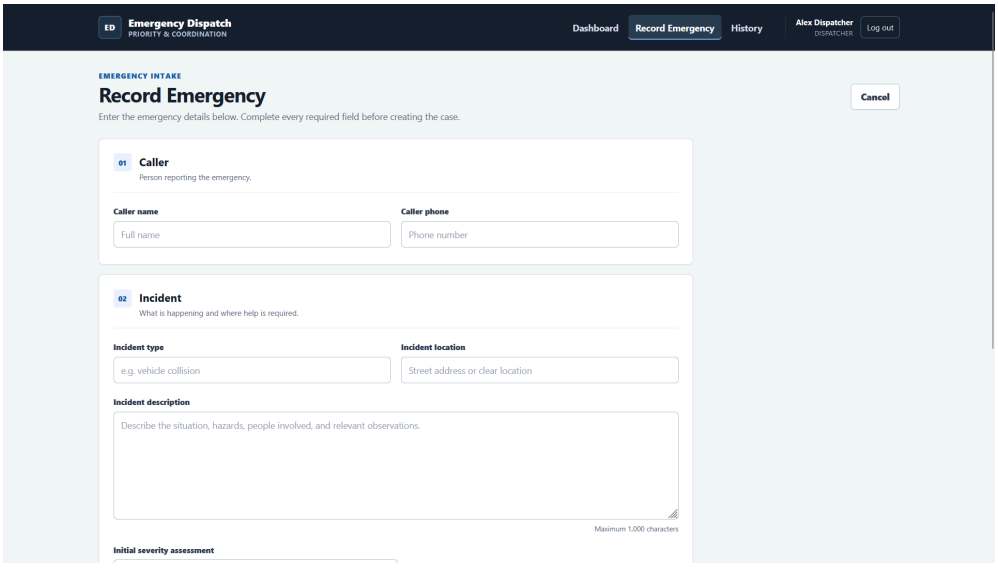


Figure 2. Record Emergency interface used by the dispatcher to create a new case.

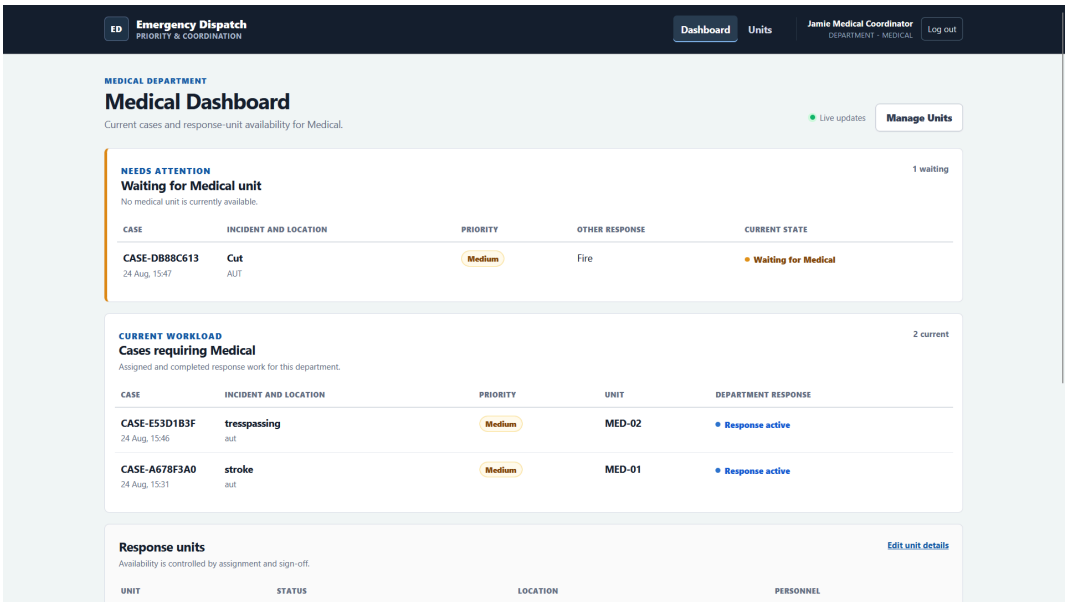


Figure 3. Medical Department Dashboard showing Medical cases, unit availability, and assignment information.

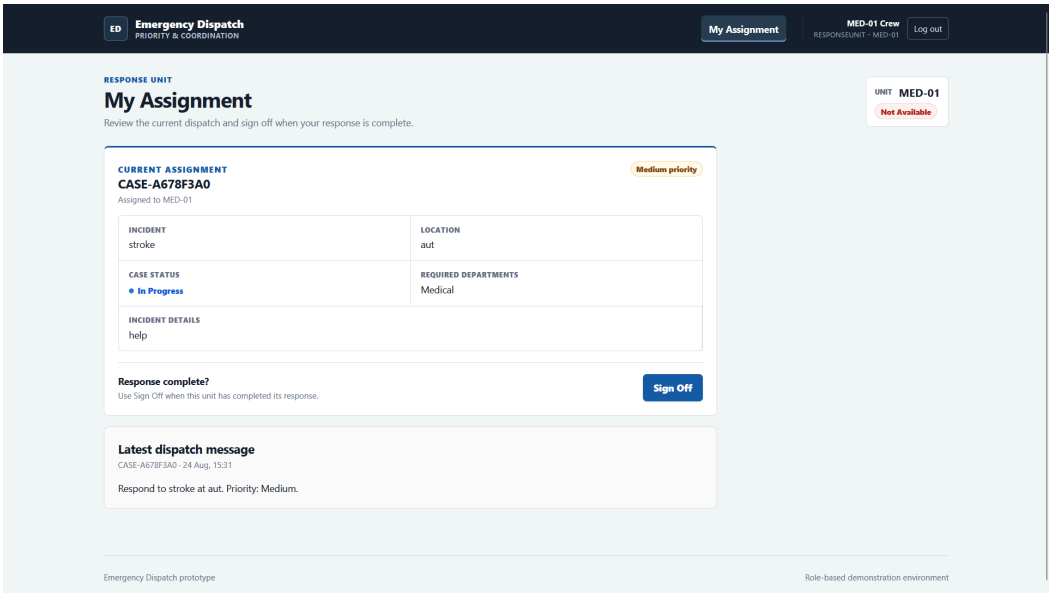


Figure 4. Response Unit interface showing the current assignment and Sign Off action.

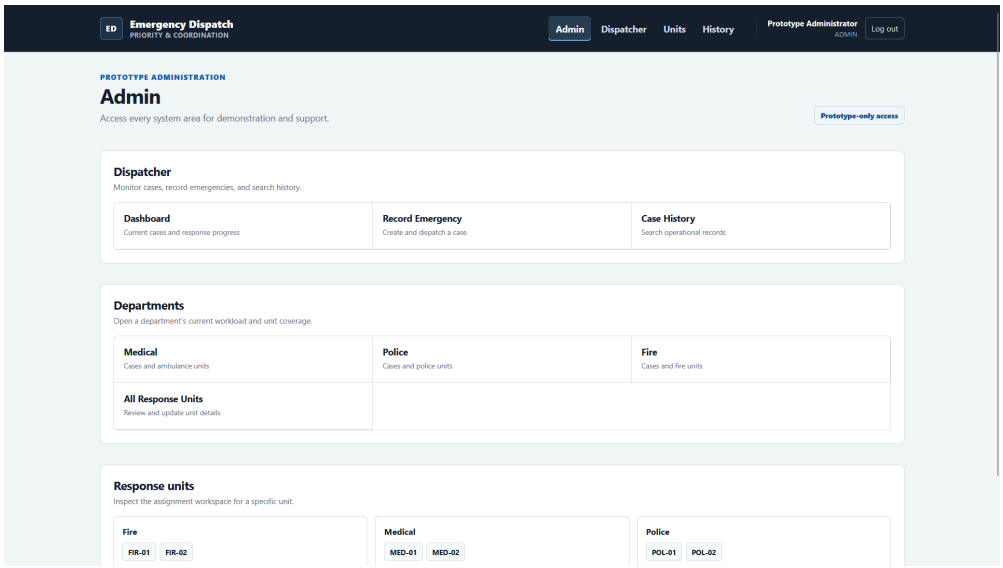


Figure 5. Prototype Admin interface providing access to all system areas.

4.5 Prototype Limitations

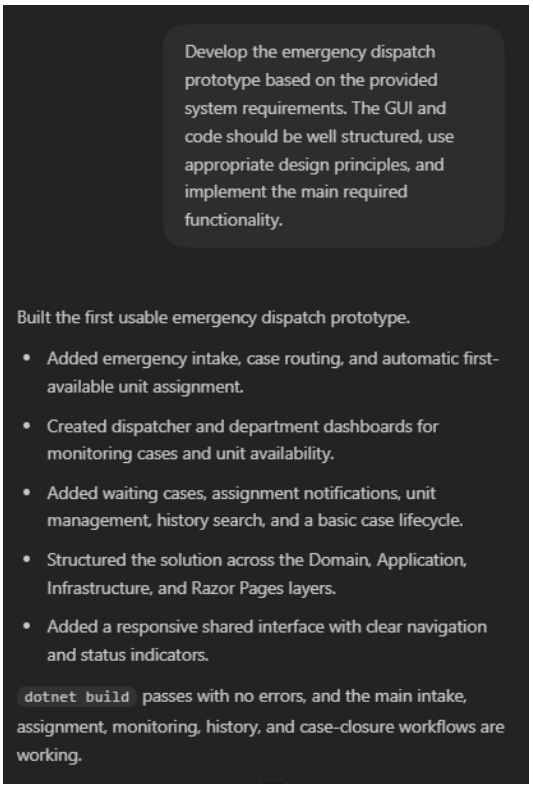
Because our current system is a prototype, there are limitations. The priority logic is a placeholder implementation that is based on keywords, e.g. stabbing, trespassing, difficulty breathing, etc. The dispatcher's selected severity is also passed through the priority determination algorithm. This is because Appendix 1 has not been completed yet.

Another limitation of our system is that there is no permanent persistence of data. The prototype stores cases, units, assignments, and notifications all into memory. When the program restarts or closes, all of that data is lost. Authentication is limited to mock plaintext accounts, which are stored in a text file as opposed to a secure database.

5. AI-Assisted Development Using GitHub Copilot

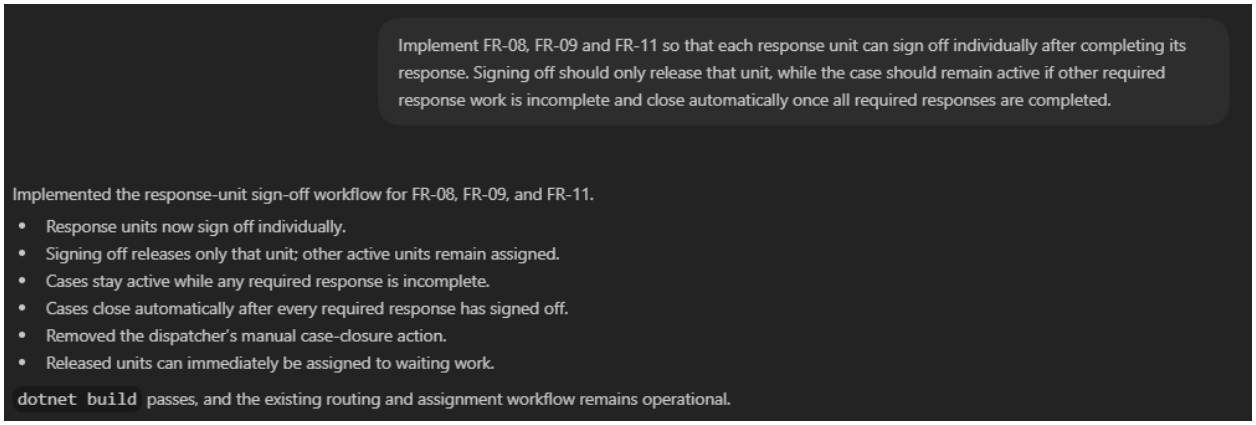
We used Codex during our development stage of the prototype. It was used to assist with implementing and improving the system features, and reviewing all the requirements. The initial prompt asked Codex to develop the initial prototype of the system based on the provided requirements *(see Appendix 5.1)*.

All of the suggestions that we used were reviewed by us before being accepted. For the suggestions that did not reflect our requirements and scope, they were either modified or rejected. The following examples show how AI was used during development, and how human review was used to ensure that the suggested system matched the project requirements.



Appendix 5.1, Initial prompt

5.1 Example one: Response unit sign off

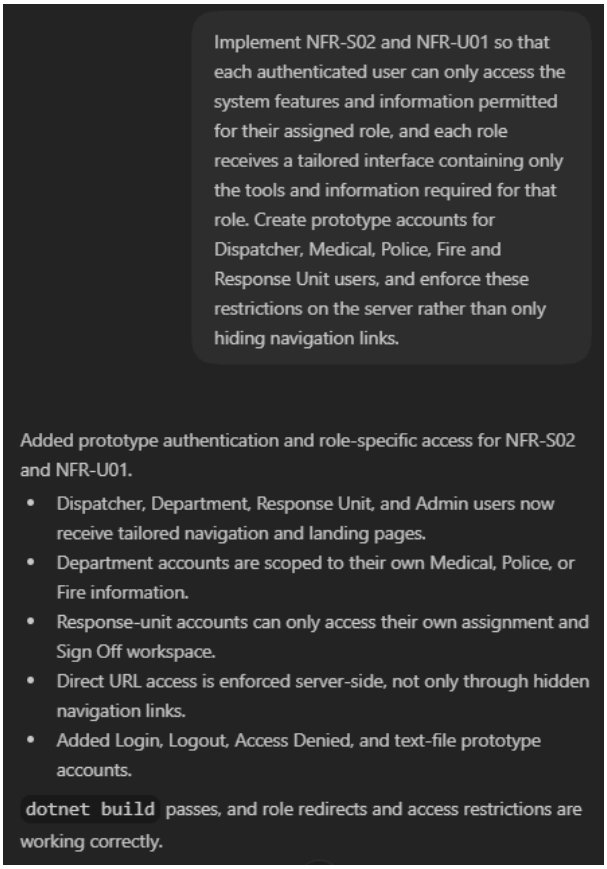


Appendix 5.2, Response unit sign off follow up prompt and Codex output

The first AI-generated implementation allowed a dispatcher to close the case manually, which released any response units assigned to that case. This was compared to the improved requirements FR-08, FR-09, and FR-11, where we realised it did not reflect our intended workflow. Each response unit was required to sign off individually after completing the assignment, not released by the dispatcher. We used a follow-up prompt with the necessary information about our requirement and intended design to correct the behaviour. Codex then updated it so that a case stays active until all response units assigned to it have signed-off. This way everyone has to complete their assignment before the case closes.

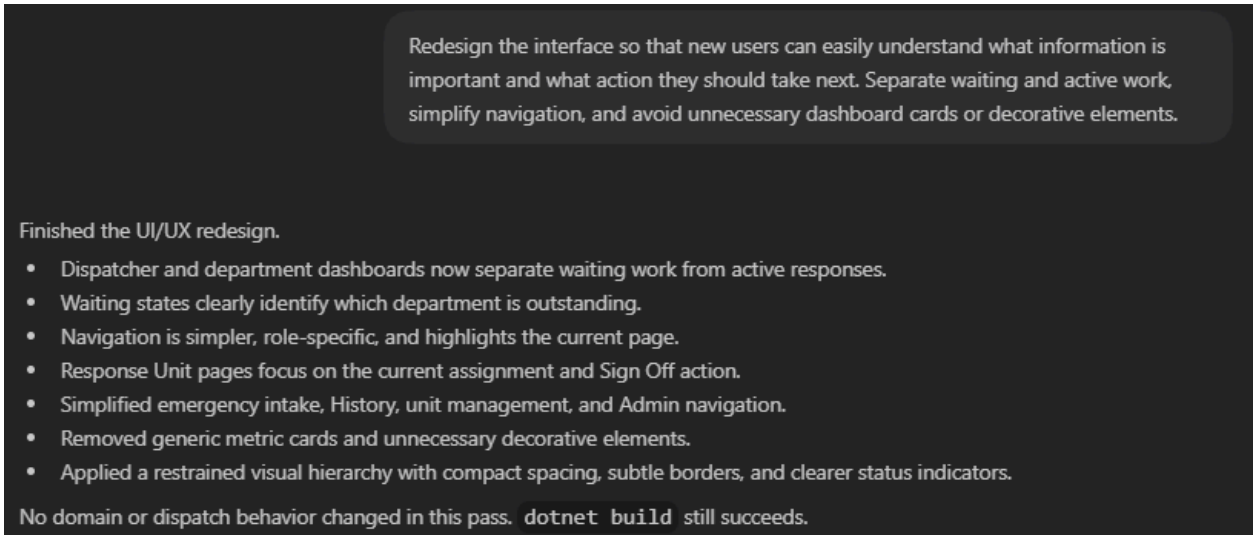
5.2 Example two: role specific access and interfaces

In the initial prototype, there were separate interfaces for different users, but no authentication or restrictions to user-based features. When comparing this prototype with NFR-S02 and NFR-U01, we realised a visual separation was not enough, and that users needed to be restricted to their features only. A follow-up prompt was used which addressed this problem and the solution to it. Codex then suggested a proper authentication process with role-specific access. The navigation was also adapted to each role, and an Admin account for testing and demonstration purposes.



Appendix 5.3, Role Access Prompt

5.3 Example Three: User Interface Redesign



Appendix 5.4, User interface redesign follow up prompt and Codex output

The AI-generated implementation provided an interface with the required functionality, but with human review, we saw several usability problems. Firstly, the waiting and active cases were displayed together, causing the interface to be too crowded and overwhelming. This raises concerns against NFR-UO3, which requires users to complete core tasks in under one hour of training. The dashboard-style summary cards were rejected as well because they did not value the actual dispatch workflow. With the next prompt, Codex separated the waiting and active cases, simplified the navigation, and focused each role's interface on their core tasks.

5.4 Evaluation of AI Assisted Development

Using AI-assisted development increased our productivity by helping with code generation and revising different parts of the prototype. All the generated implementations were assumed to not be correct, so human checking was required everytime. We reviewed the case lifecycle behaviour, role restrictions, and usability issues before accepting the AI suggestion. This shows that AI was useful for making development faster and more efficient while making sure everything was acceptable before implementation.

6. Initial Test Strategy and Test Planning

6.1 Scope of Testing

The scope of testing focus on the core functionality of the system, and quality risks that could result in incorrect information, incorrect priority management, incorrect dispatching of units, delayed response on scene, or incorrect states for units and cases.

Exhaustive testing is not logical because it is impractical to test every possible input, sequence and condition. So instead, a **risk-based testing** approach will be used.

Test Item	Requirement	Scope	Risk
Emergency call recording	FR-01	Recording required information about the caller, incident, location, emergency service types, input validation, missing information, and invalid values.	High
Case creation	FR-02	Creating a case from a completed call and verifying that all recorded information is persisted; the case receives the correct initial status, and incomplete or duplicate actions are handled correctly.	Critical
Priority determination	FR-04	Applying the priority rules for Police, Fire, and Ambulance to incident information and verifying the priority outcome, including relevant boundary conditions.	Critical
Department routing	FR-05	Routing cases to the correct department according to the emergency service types required by the case, including cases that require multiple departments.	Critical

Unit assignment	FR-06	Assigning only suitable units that belong to the correct department based on their availability. Cases with no available units remaining must wait for assignment.	Critical
Concurrent unit assignment	FR-06 / NFR-R04	Ensuring simultaneous assignments cannot allocate the same response unit to more than one case at once.	Critical
Department dashboard	FR-07	Displaying the correct department's active cases and updating case information without needing to manually refresh.	High
Case status lifecycle	FR-08	Correct transitions between Open, In Progress, and Closed, including cases where there are multiple units assigned, and cases where assigned units are removed. Invalid state transitions will also be tested.	Critical
Unit availability lifecycle	FR-09	Correct transitions between Available and Unavailable since units are assigned to and released from cases.	Critical
Response completion and sign-off	FR-11	Individual unit sign-off and automatic closure of a case, which occurs only after all units have completed their response.	High

Non-functional quality concerns will also be included in testing where it is possible to evaluate them in the current environment.

- **Reliability** can be evaluated by testing the consistency of case data during case creation, routing, assignment, and status changes.
- **Concurrency** will be tested by trying to assign the same response unit to multiple cases simultaneously.

- **Performance** can be tested by measuring the time it takes for critical operations to complete. This includes department routing and auto-refreshing of the interface.
- **Security** and authorisation can be tested using simple verification and by making sure system roles cannot access functions outside their permissions.
- **Usability** will be evaluated through basic checks of the interface, making sure error messages are clear, and whether users can perform the core tasks without asking for help.

Some areas will not receive validation during the initial test cycle. These features include:

- Large-scale testing with 100 or more simultaneous users
- Long-term verification on the 99.9% uptime requirement
- Full security testing
- Recovering after system outages
- Integration with real NZ departments
- Large-scale usability testing involving real-life stakeholders
- Overall features that are not yet implemented in the prototype, like historical log searching or advanced notification functionality.

6.2 Test Types / Levels

Test levels and types that are relevant to the Emergency Dispatch system are stated below.

- **Unit Test** - Test the individual class / methods, would be used in priority determination logic and validation of callers details through boundaries.
- **Integration Test** - Used to test different services in the system, used in testing synchronicity across different modules, like assigning a unit to a case and updating the details during assignment.
- **System Test** - Focus on testing the workflow of a process, such as recording a case from a caller to resolving the case.
- **Acceptance Test** - Ensure that the system meets the requirements set by the stakeholders.

- **Regression Test** - The modification of a section of code should not cause unwanted changes to other sections of the codebase. This is particularly useful around testing concurrency on unit assignment.

6.3 Functional testing approach

To test the functionality of the emergency dispatch system, we will determine how the system operates by checking whether the behavior of the system matches the functional requirements (section 2.4) and the acceptance criteria defined for the key features (section 2.5).

The black box techniques would be utilized as the main approach. The following black box techniques would be used in our testing.

Equivalence partitioning seperates the input into groups that are expected to behave similarly. So that all possible values don't have to be tested. For example, in FR-04, the partitions for this requirement would be split into valid values (high, medium, low) and invalid values.

Next, boundary value analysis focuses on testing the edge cases because defects often occur near the edges of valid and invalid ranges. For example in FR-06 when determining if a case needs to be in a queue or not depending on the available units, the boundaries include 0 units (case is queued), 1 unit available (case is assigned to unit) and 2 units available (first unit on the list gets assigned to the case).

A decision table would be helpful in determining the outcome of different combinations of outputs. For FR-08, where a case with multiple unit types did not sign off the case then the status would still display "In Progress", it would only become "closed" if all unit types assigned are all signed off.

The last technique is state transition testing, where it focuses on checking that the state changes are correct. This could also be used for testing FR-08.

These tests would be completed by a combination of automated testing and manual testing, depending on the requirements, as well as to double-check whether the system is working as intended.

6.4 Non-Functional Testing Approach

The non-functional requirements span over 5 categories: performance, reliability, maintainability, security and usability. Each of these categories need their own testing approach because the methods used to test one of them may not work for the others.

Timing and benchmark tests will be the approach for testing performance. For NFR-P01, the time between a case being submitted and being received from the targeted department will be measured. This test will pass if the entire flow takes less than 2 seconds. For NFR-P02, a smaller-scale test will be carried out, where we will simulate 10 concurrent users. During this, the time it takes to navigate pages or refresh the page will be measured and recorded. The smaller-scale test will be used for the prototype because testing 100 concurrent users is out of scope for the prototype, as noted in section 6.1.

Reliability testing will be on data consistency and concurrency. For NFR-R01, the input from the dispatcher will be compared to the information received from the targeted department. If they match, then we can verify that no data was lost throughout the operation. For NFR-R03, the submitted case will be compared to the stored records in the database. This will ensure that all data is persisted correctly and that no data was lost during persistence. NFR-R04 will be tested using race-condition testing. This is where multiple case assignments are called simultaneously to verify that a single unit cannot be assigned to multiple cases at once. NFR-02 will not be tested because that requires our system to run for a long-period of time, and it is out of scope for the prototype.

For maintainability testing, there will be no test cases that we check for pass or fail. Instead, it will be reviewed by our team and checked against Appendix 2, which will tell us what our system architecture will be like. For NFR-M02, the design will be reviewed by our team to determine whether a new emergency department or user role can be added without requiring lots of modification to an existing class. These reviews will determine whether our system follows the modular design we planned.

Security isn't a big focus in the prototype, so only small security testing will take place. For NFR-S02, encryption will have to be implemented so that we can test if data is encrypted properly. Advanced security testing will not occur for now.

For usability testing, only basic checks will be made by using the usability requirements. For NFR-U02, the displayed components for each user role will be checked to see if the right tools are loading on the user's screen. For NFR-U02, errors will purposely be implied to check that error messages are working and provide clear instructions to help the user. NFR-U03 will not be tested in the prototype because that requires users to test the system for an hour; this is out of scope for now.

6.5 Entry and Exit Criteria

The entry criteria make sure the system is ready to produce meaningful test results before testing can be started. This is when the development team agrees the system has reached the acceptable level of quality. Testing can begin when all of the criteria are met. If any of these conditions aren't met, the testing will be delayed due to incomplete features or an incorrect test environment. If this is the case, the developer will not know the source of the error.

Number	Criteria
1	The prototype should build successfully without compile errors.
2	The feature being tested is implemented and available in the testing environment
3	Relevant functional and non-functional requirements should be defined
4	Acceptance criteria for the functional requirements should be defined
5	The required test data should be prepared
6	Expected results should be defined for each planned test case

Figure x: Entry Criteria table

For the testing phase to be complete, the exit conditions should be met. These criteria do not mean that the prototype is ready for real-world use; it just verifies that the prototype testing is complete and enough evidence has been collected.

Number	Criteria
1	All Critical and High-risk tests that were planned have been executed
2	All acceptance criteria for the selected features have been tested
3	There are no unresolved, critical-severity defects
4	Any unresolved high-severity defects have been reviewed and documented
5	All critical workflows are working according to their expected results
6	Expected results should be defined for each planned test case
7	The concurrency test confirms the same unit cannot be assigned to multiple units simultaneously
8	Any test results, failed tests, and unresolved defects have been documented
9	Some requirements will not be tested in the prototype stage; these should be documented as limitations.

Figure x: Exit Criteria table

6.6 Test environment, tools, and dependencies

Test Environment

The testing for the system will be carried out in a local development and testing environment. The application is developed using C# and .NET for the backend, and HTML/CSS is used for the user interface. Visual Studio & Visual Studio Code is used as the main development environment for building, debugging, and testing all code.

We will use MSTest to create and run both unit and integration tests. This will allow us to test repeatable logic like priority determination, department routing, unit assignment, and state changes.

For version control, we will use a shared repository on GitHub. This will allow us to work on the system and collaborate more easily, and store backups of our system code.

For any testing, mock data will be used rather than real emergency information. This means we will use fictional callers, cases, departments, and units with different states.

Our prototype is going to be tested on computers through the browser. The operating system and browser used during testing will need to be recorded so that any defects can be reproduced under the same conditions and later fixed.

7. Initial Test Cases and Requirements Traceability

The designed test cases were created using the improved requirements from section 2.4, acceptance criteria in section 2.5, and the risk-based testing scope in section 6.1. The test cases focus on the system operations with the greatest potential impact being: recording case information, routing cases, assigning units, and maintaining case and unit states. Non-functional requirements are more complicated to test, so they will be included wherever they can be evaluated in the prototype environment.

7.1 Initial Test Cases

TC-01: Record a complete emergency call

Field	Details
Requirements	FR-01
Test level/type	System and acceptance test / positive test
Risk priority	High
Preconditions	The prototype is running, and the dispatcher has authorisation on the information recording interface.
Test data	Caller: John Smith, phone: 021 123 4567, incident type: vehicle collision, location: 25 Queen Street, description: two vehicles blocking the road, reported severity: High, required unit types: Police and Medical.
Steps	1. Enter all details from the call 2. Select Police and Medical as required unit types 3. Click "create and dispatch case"
Expected result	One case is created and given a UUID with a recorded timestamp. Every input is saved without data alteration. The system determines the priority, makes the case available, and assigns the suitable units.

TC-02: Rejecting an incomplete emergency call

Field	Details
Requirements	FR-01, NFR-U02
Test level / type	System and acceptance test / negative test and equivalence partitioning
Risk priority	High
Preconditions	The dispatcher is on the call-recording interface
Test data	Correct data for every input except for the blank location. Another case with correct data for every input, but the required unit type isn't selected.
Steps	1. Enter the test data 2. Submit the record with the location blank 3. Observe the response 4. Enter the location, remove all selected unit types, and submit again
Expected result	Both submissions are rejected, and no record is created. The missing field is identified using a clear error message, and it tells the dispatcher how to correct it. No exception, stack trace or debug information should be displayed.

TC-03: Create and dispatch a case

Field	Details
Requirements	FR-01, FR-02, FR-04, FR-05, FR-06, FR-08, FR-09
Test level / type	Integration and system test / end-to-end positive test
Risk priority	Critical
Preconditions	The prototype is running. The police and medical departments exist, with one available unit in each. The dispatcher must be on the case-creation interface.
Test data	Caller: John Smith; phone: 021 123 4567; incident type: vehicle collision; location: 25 Queen Street; description: two vehicles blocking the road; reported severity: High; required departments: Police and Medical.
Steps	1. Enter call details 2. Record the time before submission 3. Click "create and dispatch case" 4. Open dispatcher dashboard 5. Open Police and Medical dashboards 6. Compare the displayed case fields with submitted form values 7. Check the assigned units and availability
Expected result	The submission creates only one case with a UUID and recorded timestamp. All submitted inputs are preserved. The system calculates the priority given to both departments. And assigns the first suitable units. Those units should become "not available", and the case should be "in progress".

TC-04: Determine priority using approved department rules

Field	Details
Requirements	FR-04
Test level / type	Unit and acceptance test / decision-table testing
Risk priority	Critical
Preconditions	The priority decision processes in Appendix 1 are approved. There is a new case available for processing.
Test data	A representative Medical, Police, and Fire case for each priority outcome is defined in Appendix 1. Each data row must have a predetermined expected priority.
Steps	1. Submit each case to the priority component 2. Record the returned priority 3. Compare it with the outcome defined by the relevant department decision process
Expected result	Every case is assigned a priority automatically, with no input. Each result matches the approved decision process for its department, also including the defined boundary cases.

TC-05: Routing a multi-department case within 2 seconds without data loss

Field	Details
Requirements	FR-05, NFR-P01, NFR-R01
Test level / type	Integration and performance test / timing and data-consistency test
Risk priority	Critical
Preconditions	Police, Fire, and Medical department endpoints are available. A timing tool to record the time it takes for routing.
Test data	A complete case requiring all units.
Steps	1. Capture a copy of every case field 2. Start timing when the completed case is submitted for routing 3. Stop timing when all relevant departments receive it 4. Compare received copies with the submitted case 5. Check that no unrelated department received it
Expected result	The case is delivered to the Fire, Medical, and Police departments within 2 seconds. No unrelated department receives it. Every received case has the same details as the submitted case.

TC-06: Assign only an available unit and select the first available unit

Field	Details
Requirements	FR-06, FR-08. FR-09
Test level / type	Unit and acceptance test / boundary value test
Risk priority	Critical
Preconditions	A Police case has been routed to the Police department. Unit list:: P-01: unavailable, P-02: available, P-03: available.
Test data	One open Police case, and the units list as described above
Steps	1. Trigger the automatic unit assignment 2. Inspect the assigned unit 3. Inspect the state of the case and all three units.
Expected result	P-02 is assigned; the rest are not assigned. P-02 changes to "not available", the case changes from "open" to "in progress".

TC-07: Queue a case when no units are available

Field	Details
Requirements	FR-06, FR-08, FR-09
Test level / type	Unit and integration test / boundary value test
Risk priority	Critical
Preconditions	A Fire case is routed to the Fire department. All Fire units are marked as "not available"
Test data	One open Fire case, and zero available Fire units
Steps	1. Trigger the automatic unit assignment 2. Inspect the case queue and status 3. Change the first fire unit to available 4. Process the queue again
Expected result	At first, no unit is assigned, and the case remains "open". When a unit is available, the queued case is assigned to that unit. The case becomes "in progress", and the unit becomes "not available"

TC-08: Prevent same unit from being assigned to two cases at once

Field	Details
Requirements	FR-06, FR-09,
Test level / type	Integration and regression test / concurrency test
Risk priority	Critical
Preconditions	Only one suitable Ambulance unit, A-01, is "available". Two open Medical cases are ready for assignment. Two assignments are released simultaneously
Test data	Case C-101 and C-102, unit A-01
Steps	1. Prepare the assignment requests for C-101 and C-102 2. Release both requests at the same time 3. Wait for both operations 4. Inspect both cases, A-01, and assignment records 5. Repeat the test to detect race conditions
Expected result	A-01 is available to only one case and has only one active assignment record. Its status is "not-available". The other case remains "open" and in the queue. The system accepts and displays both cases without corrupting their data.

TC-09: Display and automatically update the department dashboard

Field	Details
Requirements	FR-07
Test level / type	System and acceptance test / positive and separation test
Risk priority	High
Preconditions	The Police dashboard is open. Existing Police and Fire cases are available
Test data	Police case P-C01, Fire case F-C01, and a new Police case P-C02
Steps	1. Open the police dashboard 2. Verify the initial cases 3. Route R-C02 to the Police Department 4. Change the priority or location of P-C01 with another valid session 5. Observe the dashboard without manually refreshing
Expected result	The dashboard displays P-C01 and P-C02 but not F-C01. P-C02 appears automatically with no manual refresh required. The changes for P-C01 is shown automatically as well.;

TC-10: Keep a multi-unit case open until the final unit signs off

Field	Details
Requirements	FR-08, FR-09,. FR-11
Test level / type	Integration and acceptance test / decision-table and state-transition test
Risk priority	Critical
Preconditions	A case is "in progress" with Police unit P-02 and Ambulance unit A-01 assigned. Both units are "not available".
Test data	One active multi-department case with two assigned units
Steps	1. Sign off P-02 2. Inspect the case and both unit states 3. Sign off A-01 4. Inspect the final case and unit states
Expected result	After the P-02 unit signs off, it becomes "available", but the case is still "in progress" because the A-01 unit hasn't signed off. After A-01 signs off and becomes "available", the case automatically becomes "closed".

TC-11: Return a case to Open when all assigned units are removed

Field	Details
Requirements	FR-08, FR-09
Test level / type	Integration test / state-transition and negative-path test
Risk priority	Critical
Preconditions	A case is "in progress"; one unit is assigned, and the unit hasn't been completed or signed off.
Test data	Case C-103 and assigned unit P-03
Steps	1. Remove P-03 from the case before response completion 2. Inspect the case, unit and assignment record
Expected result	After the active assignment is removed, the P-03 unit becomes "available", and the case returns to "open" rather than becoming closed. The case can be queued or reassigned.

TC-12: Enforce role-based access to features

Field	Details
Requirements	NFR-S02, NFR-U01
Test level / type	System test / basic authorisation and usability test
Risk priority	High
Preconditions	The role permissions in Appendix 3 have been completed and implemented. Test accounts exist for a dispatcher, department user, and response unit user.
Test data	One authenticated account for each role
Steps	1. Sign in as each role 2. Record the visible pages, tools and case information 3. Attempt to open one URL or action that Appendix 3 does not allow for that role
Expected result	Each role only sees its permitted tools and information. An unauthorised page or action is denied, and the denial does not expose protected data or error details.

7.2 Initial Requirements Traceability Matrix

Requirement	Summary	Linked test cases	Coverage / current status
FR-01	Record all required call details	TC-01, TC-02, TC-03	Positive input, missing input, persistence during submission
FR-02	Create a case when details are submitted	TC-03	After creation, UUID and timestamp are stored correctly.
FR-03	Department management of response-unit details	None	Out of testing scope
FR-04	Automatically determine priority using department rules	TC-03, TC-04	Submission and decision-process outcomes and boundaries. Final decision data depends on Appendix 1
FR-05	Route to every relevant department	TC-03, TC-05	Submission integration, department routing, exclusion of irrelevant departments
FR-06	Assign the first suitable unit or queue the case	TC-03, TC-06, TC-07, TC-08	Submission integration available/unavailable boundaries, concurrent assignment
FR-07	Show and automatically update department cases	TC-09	Different interfaces for departments, Displays cases automatically
FR-08	Valid “open”, “in progress”, and “closed” states	TC-03, TC-06, TC-07, TC-10, TC-11	Main lifecycle transitions and removal path
FR-09	Unit availability should match assignment and release	TC-06, TC-08, TC-07, TC-10, TC-11	Assignment, sign-off, and removal paths

FR-10	Notify the assigned unit on the dashboard	None	Out of testing scope
FR-11	Permit unit sign-off and close after final sign-off	TC-10	Partial and final sign-off for multi-unit case
FR-12	Search historical logs	None	Out of testing scope
NFR-P01	Route a completed case within 2 seconds	TC-05	Executable timing measurement
NFR-P02	Support 100 concurrent users	None	Out of testing scope
NFR-M01	Isolate core functions from less important modules	Design/code reviews	Reviewed against Appendix 2
NFR-M02	Add roles/departments without modifying existing classes	Design/code reviews	Reviewed against Appendix 2
NFR-R01	Preserve case data during routing	TC-05	Field-by-field sender/receiver comparison
NFR-R02	Maintain 99.9% available over a week	None	Out of testing scope
NFR-R03	Save reports and case-log data without loss	None	No permanent persistence
NFR-R04	Continue operation with 100 open cases	None	Out of testing scope
NFR-S01	Encrypt data passing through the system	None	Out of testing scope
NFR-S02	Restrict features and information by role	TC-12	Basic interface and direct-access authorisation checking

NFR-U01	Each user role gets customised interface	TC-12	Visible tools and information checked against Appendix 3
NFR-U02	Display plain-language errors that are easy to follow	TC-02	Validation error content and no technical details
NFR-U03	Complete core tasks after no more than one hour of training	None	Out of testing scope

7.3 How Traceability Supports Quality Assurance and Change Management

The requirements traceability table provides a clean view for the team that shows the connections between requirements and testing. It shows how every requirement from section 6 is linked with one or more test cases and that the acceptance criteria was met for each one. This table supports quality assurance by making the test coverage visible to the team, which helps them prioritise any critical tests.

Requirements traceability helps view and manage changes to a requirement. If there was a change, the row would display the test-cases linked to it, which can be reviewed and retested. A failed test could also be traced back to the requirement that was affected and its acceptance criteria. This makes defect reporting and analysis easier for the development team.

The traceability matrix should always be maintained throughout the entire development process, because it helps a lot during the software testing life cycle. When all the Appendices are completed and agreed on, the other requirements that rely on it, will be added to the traceability matrix with their coverage and linked test-cases. All results and notes should also be documented so that the team can go back and use it to review any test-cases that need to be retested.

8. Project Progress, Risks, and Next Steps

8.1 Summary

In this report Auckland City Council have found a need for a better emergency dispatch system to replace the legacy system. The current legacy system does not meet the 3 quality attributes (reliability, usability, or maintainability) which this prototype would aim to improve on. The system involves many manual processes which compromise the effectiveness and efficiency of the system. The proposed solution is to create a standardized priority allocation and allow the system to route cases automatically.

From the proposed solution and the problem statement, a set of functional and non-functional requirements have been developed to have an organised way of developing and testing the prototype to ensure that all requirements are considered. The requirements are then reviewed to remove ambiguity and improved clarity to ensure that the features developed are testable. Acceptance criteria were also developed for key features to ensure that the important features have a clear guideline of what needs to be included. Once the requirements were completed, the initial prototype was developed. The prototype mainly focuses on the core functionality which includes the creation of a case from the dispatcher. Throughout the development of the prototype there were several limitations identified which included a placeholder for the priority logic and the lack of a database.

The next step performed after the development is the early phase of the software testing life cycle, where the test strategy and test planning have been completed. This would provide a clear way to identify and solve potential issues when testing for the system. These steps through the report provide a path to developing a prototype that meets software quality attributes and functionality.

8.2 Current Limitations, Risks, and Challenges

As stated in section 4.5, the main limitation during the development of the initial prototype does not have an industry standard prioritization algorithm. The current algorithm utilizes a limited amount of keywords to determine certain priority therefore it is limited based on that. Another limitation of the prototype is the in memory storage of the current accounts, and units types. Discussed in section 4.5.

The priority algorithm is also a risk because using keywords to identify the incident details could fail which would affect selecting the appropriate priority level. The system would need to also consider the scalability as stated in the section 6.1, because there is no verifiable way to ensure that the system can handle 100 or more concurrent users . This is important since the emergency dispatch system would need to be accessed frequently and in huge volumes. Further stated in the limitations that we currently use in memory storage, this would mean that the case created and assigned to a unit would be lost when closing the system.

Some challenges we faced during this phase of the project was coordinating the system architecture. We each had a different idea of how to build our system architecture, so we needed to have a meeting and agree on one. Using github was another issue we faced early on. There were some issues with conflicts due to two people working on the main branch instead of separate branches. The last challenge we faced was linked to testing and reviewing each other's code. Testing is a new topic, so we each had a different level of knowledge leading to some misunderstandings.

8.3 Managing Risks

To manage the above mentioned risks, we would first improve on the priority system by researching and developing the standard protocol used in determining priority based on actual systems. For managing concurrent use at a large scale, we could not simulate the testing of 100 or more users due to time constraints. Therefore instead we should perform a small scale concurrency test of around 10 requests at a time to verify that it could handle a good number of concurrent access. To prevent data loss caused by the use of in memory storage, simply implementing a database would be necessary.
The current testing has passed testing but yet it is not ready for the release of the system. The next phase of this project will use a risk-based approach, meaning that features with the highest risk will get the most attention. These will be reviewed throughout development, and worked on until they are no longer able to occur or cause bugs in the system.

8.4 Future Plans

The next phase will focus on the remaining features and current incomplete features that have yet to be completed. All of these features will be tested in different amounts, depending on their risk-factor and contribution to the project. Some features like the priority scheduler and case assignment will focus heavily on during the testing phase, so that any risks can be minimised and avoided. All current test cases that have failed or are incomplete have been documented, and will also be focused on and finished before new features are implemented..

The testing plan and strategy will be updated to include new features that may be added to bring up any problems that may occur. Any defects that occur during the testing phase will be classified according to severity and then linked to their associated requirements. Any critical defects will need to be sorted out before the project can be considered complete, and the progress should be documented for later review.

The requirements traceability matrix will also need to be updated with the testing results and defects that appeared. Any data relating specifically to the prototype will be removed so that the requirements, acceptance criteria, test cases, and implemented behavior of the system remain consistent.

Before this system can be considered ready for real emergency use, these things must be complete and tested. There has to be a secure database solution to fix the problem of temporary memory, and security when it comes to accounts and user-based roles. All the features should be fully tested and documented to show its functionality and proof that it is ready for real-world use.

The last thing before the system can be considered complete is a test with stakeholders. It needs full-scale performance (multiple concurrent users), availability, security, recovery, and usability testing. Everything that could not be tested in the prototype must be a priority during the next phase and before this system can be deployed.

8.5 Team Reflection

Overall, utilizing AI assisting learning was crucial in the development of the emergency dispatch system. It was helpful in making sure that the stated requirements are met in the prototype, as well as ensuring that all crucial test cases and boundaries are covered. In the group project we had many conflicts on how the system should be designed. During the requirements planning, we had different understandings on what the system should do, and nothing was aligned. To get back on track, we had several meetings which helped all of us get the same understanding of our system and what it does. Throughout the whole of phase 1, everyone contributed

Team Member	Contribution	Collaboration
Shawn Lee (23204035)	33.33%	• Section 1 (wrote part of the problem statement) • Section 2 (Creating Functional requirements, Analysis of functional requirements, Improved Requirements, and create some acceptance criteria) • Section 6 (Wrote test types, and the functional testing approach, Environment + Tools) • Section 8 (Summarise report, identify limitations and risk and how to manage them)
Fateh Bhular (23217987)	33.33%	• Section 1 (Problem statement, Why the problem is suitable) • Section 2 (Non-functional requirements, Analysis of requirements, Acceptance criteria) • Section 5 (Evaluation of AI-assisted development) • Section 6 (Scope of Testing, Non-Functional Testing approach, Entry/Exit criteria) • Section 7 (Test cases, Traceability Matrix)

Leo Cao (23211788)	33.33%	• Section 1 (Background, problem statement) • Section 2 (Functional and non functional requirements. Analysis and improved requirements.) • Section 3 (Proposed solution) • Section 4 (Initial prototype) • Section 5 (Example 1, 2, and 3, evaluation)

Github: https://github.com/ShawnLeeyz/Emergency-Dispatch-Priority-and-Coordination-System.git